Pre-Build Pipeline Workflow

1. **Code Push to SCM**
   The source code is pushed to the Source Code Management (SCM).

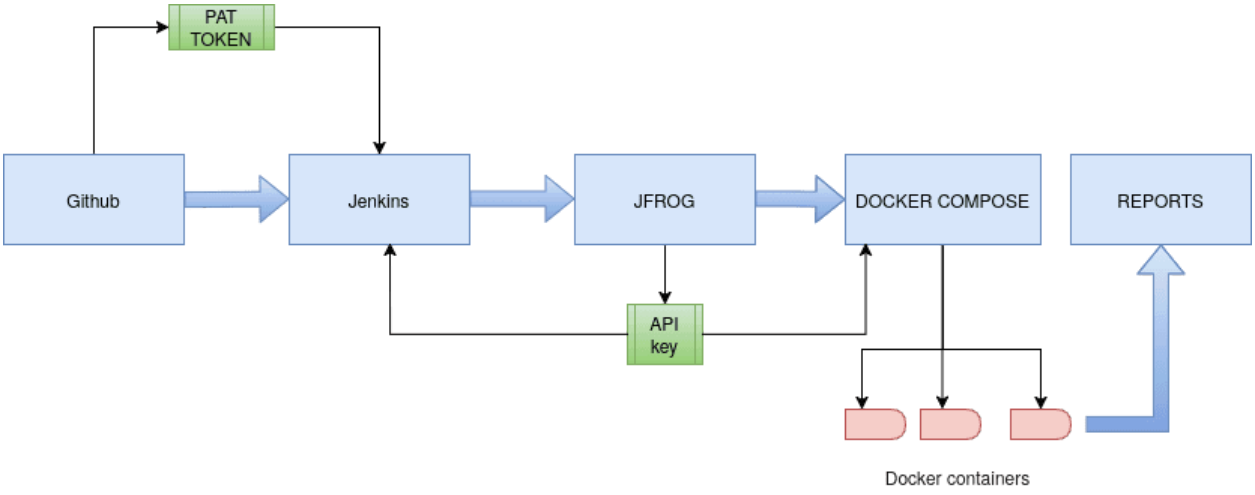
2. **Build Triggered by Jenkins**
   Jenkins triggers the build process, which includes compiling the code and packaging it.

3. **Upload to JFrog Artifact**
   Once the build is complete, the compiled binary is uploaded to **JFrog Artifact** for storage and later use.

4. **Container Deployment via Docker Compose**
   Container hosts pull the compiled binary from JFrog Artifactory and launch application containers using **Docker Compose**.

5. **Test Execution and Reporting**
   Inside the running containers, the Report.sh script executes the required tests.
   The test results are then sent back to the **Jenkins controller** for review and reporting.

Basic workflow diagram



Jenkins Host Installation Guide

1. Prerequisites

Install Java version 17 and Git:

```
apt install openjdk-17-jdk git
```

2. Set Up Proxy (if required)

Export the proxy environment variables:

```
export http_proxy=http://192.168.100.100:3128
export https_proxy=http://192.168.100.100:3128
```

3. Download and Install Jenkins

Download the Jenkins .deb package:

```
wget    https://pkg.jenkins.io/debian-stable/binary/jenkins_2.492.3_all.deb
```

Install the downloaded package:

```
apt install ./jenkins_2.492.3_all.deb
```

Check the Jenkins service status:

```
systemctl status jenkins.service
```

4. Create Build User

Create a new user named build:

```
adduser build
```

Grant sudo access by editing the sudoers file (/etc/sudoers) and adding the following line:

```
build ALL=(ALL) NOPASSWD: ALL
```

Add the build user to the jenkins group:

```
usermod -aG jenkins build
```

5. Configure Jenkins to Run as Build User

Edit the Jenkins service configuration file:

```
vim /lib/systemd/system/jenkins.service
```

Modify the following lines:

```
User=build
Group=build
Environment="JENKINS_HOME=/home/build/jenkins/"
WorkingDirectory=/home/build/jenkins/
```

6. Apply Changes and Restart Jenkins

Reload systemd configuration:

```
systemctl daemon-reload
```

Stop Jenkins:

```
systemctl stop jenkins
```

Start Jenkins:

```
systemctl start jenkins
```

7. Access Jenkins Web Interface

Open a browser and navigate to http://<your-server-ip>:8080

To get the initial admin password, run:

```
cat /home/build/jenkins/secrets/initialAdminPassword
```

Use this password to log in, create the first admin user, and start using Jenkins.

Jenkins Plugin Installation

Navigate to:

Manage Jenkins → Plugins → Available Plugins

Search for and install the following plugins:

- git
- github
- github api
- git parameter
- Docker Pipeline
- Docker Compose Build Step
- Docker
- JFrog
- Artifactory Client API

- Artifact Manager
- Pipeline: Stage View
- Pipeline Utility Steps
- Build Pipeline
- command agent launcher
- build timeout

JFrog Host Installation

1. Install Java (required to run Artifactory)
Install Java 17:

```
apt install openjdk-17-jdk
```

2. Set Up Proxy (if required)
Set proxy environment variables:

```
export http_proxy=http://192.168.100.100:3128
export https_proxy=http://192.168.100.100:3128
```

3. Download and Install JFrog Artifactory
Download the .deb package:

```
wget https://releases.jfrog.io/artifactory/artifactory-debs/pool/jfrog-artifactory-oss/jfrog-artifactory-oss-[RELEASE].deb
```

Install the package:

```
apt install ./jfrog-artifactory-oss-[RELEASE].deb
```

4. Pre-Start Configuration (Important!)
Before starting the Artifactory service, set the allowNonPostgresql parameter to true in the following file:

/opt/jfrog/artifactory/var/etc/system.yaml

```
************ SUCCESS ****************
The Installation of Artifactory has completed successfully.

Note: We recommend that you use Artifactory with an external PostgreSQL database.
      For details about how to configure the database, refer to
      https://jfrog.com/help/r/jfrog-installation-setup-documentation/postgresql-for-artifactory

      To run Artifactory with any database other than PostgreSQL (embedded Derby,
      MySQL, Oracle, Microsoft SQL Server and MariaDB),before running Artifactory,
      you are required to set the allowNonPostgresql parameter as true.

      For more information, refer to
      https://jfrog.com/help/r/jfrog-installation-setup-documentation/choose-the-right-database

Start Artifactory with:
> systemctl start artifactory.service

Check Artifactory status with:
> systemctl status artifactory.service
```

systemctl start artifactory.service

systemctl status artifactory.service

```
root@172.20.99.179~#systemctl status artifactory.service
● artifactory.service - Artifactory service
     Loaded: loaded (/lib/systemd/system/artifactory.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-04-20 12:26:52 IST; 3min 48s ago
    Process: 3598978 ExecStart=/opt/jfrog/artifactory/app/bin/artifactoryManage.sh start (code=exited, status=0/SUCCESS)
   Main PID: 3605836 (java)
      Tasks: 0 (limit: 38308)
     Memory: 56.0K
     CGroup: /system.slice/artifactory.service
             ‣ 3605836 /opt/jfrog/artifactory/app/third-party/java/bin/java -Djava.util.logging.config.file=/opt/jfrog/art
lines 1-9/9 (END)
```

Access JFrog Artifactory

Open a browser and enter your server IP with port 8081:

http://<your-server-ip>:8081

Login using the default JFrog credentials:

User:admin
Password:password

You can set a strong password after the first login.

Container Host Installation

Install Docker and required components:

apt install docker docker-compose docker.io docker-registry

Integration and Configuration

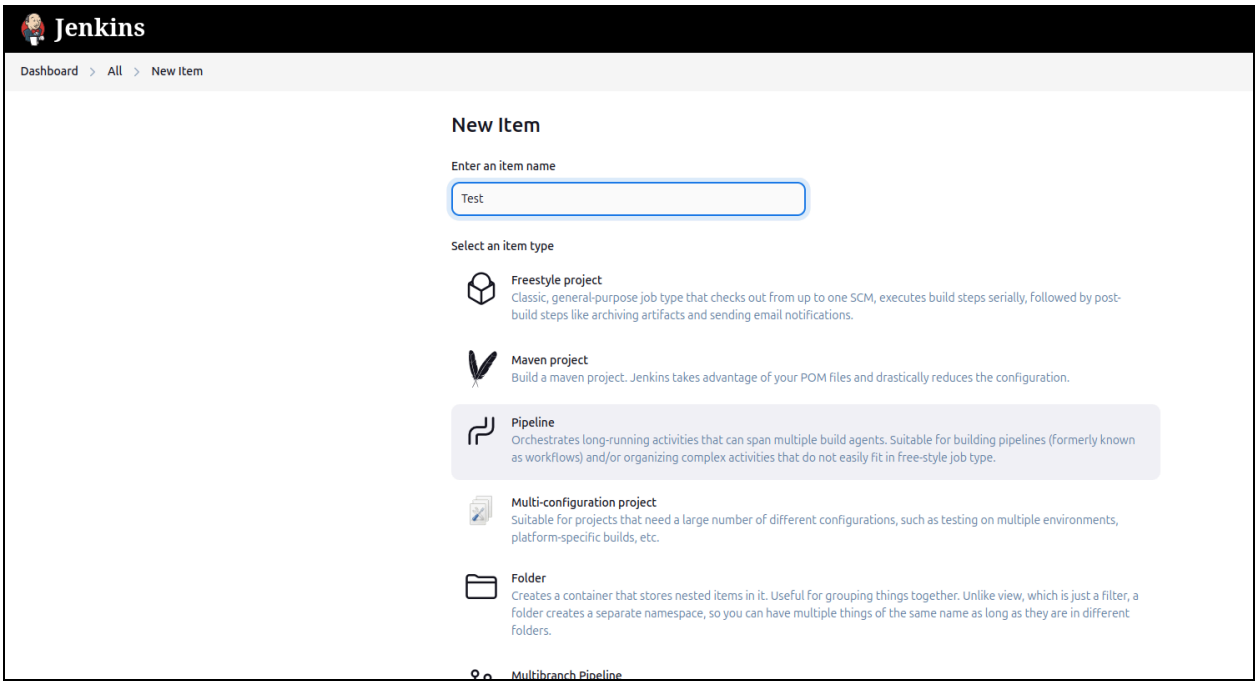
Jenkins Pipeline Setup

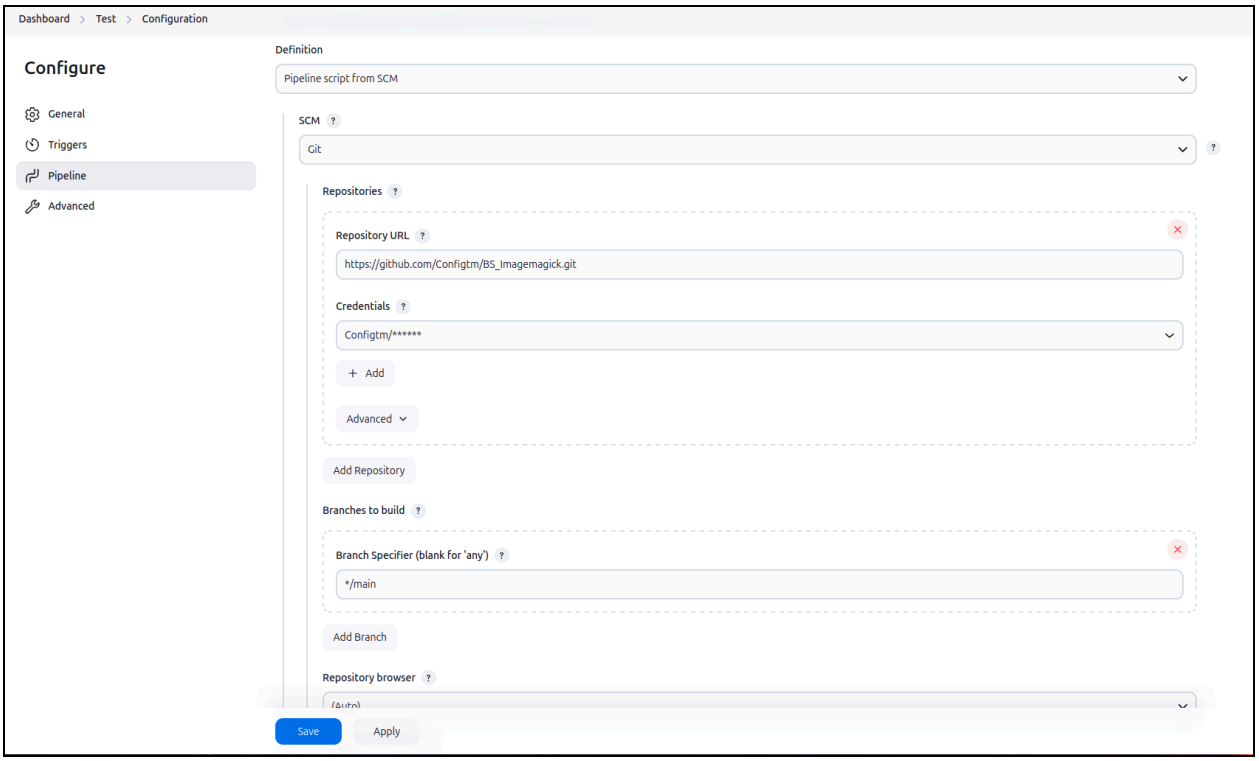
Start by creating a new pipeline:

1. Navigate to the Jenkins Dashboard
2. Click on **New Item**
3. Select **Pipeline** and provide a name
4. In the pipeline settings:
    - Choose **Pipeline script from SCM**
    - Set **SCM** to **Git**
    - Enter the **GitHub repository URL**
    - Add your **credentials**
    - Select **branch** as main

Note: We will configure secrets later in this document.

Click **Apply** to save the configuration.





JFrog Repository Setup

1. Login to JFrog Web UI

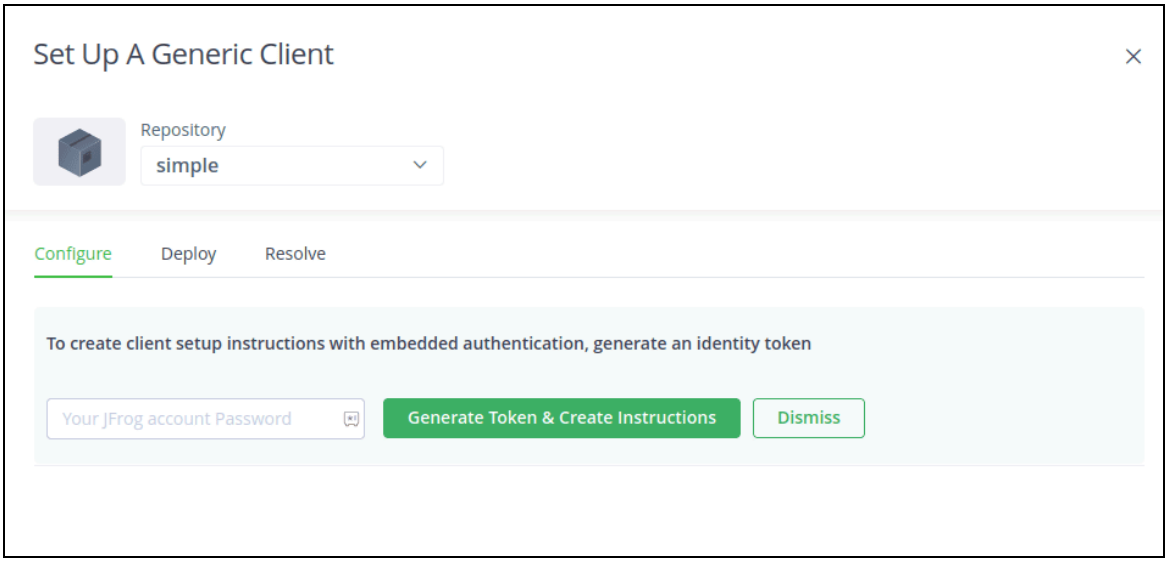
- Open your browser and go to: http://<your-server-ip>:8081
- Login with your JFrog credentials.

2. Create a Local Repository

- In the JFrog Web UI, go to **Administration → Repositories**
- Click on **Create a Repository**
- Select **Local** as the repository type
- Choose **Package Type** as **Generic**
- Set a **Repository Key** (provide a unique name)
- Click **Create Repository**

3. Generate JFrog API Key

- In the top-right corner, click on your **profile icon**
- Select **Set Me Up**
- Choose the repository you created
- Enter your JFrog password to generate an **API Key**



GitHub – Jenkins Integration

1. Generate GitHub Personal Access Token (PAT)

- Go to your GitHub profile
- Navigate to **Developer Settings → Personal Access Tokens → Tokens (Classic)**
- Generate a new token with the required scopes

You have now created two credentials: **username** and the **generated PAT (password)**. These will be used to allow Jenkins to connect to GitHub.

2. Add GitHub Credentials to Jenkins

- Login to the Jenkins Web UI
- Navigate to:
  Dashboard → Manage Jenkins → Credentials → System → Global Credentials → Add Credentials
- Select **Username and Password**
  - **Username:** your GitHub username
  - **Password:** your GitHub Personal Access Token
  - **ID:** provide a unique ID (you'll use this in your pipeline script)

Click **OK** to save.

Docker – Jenkins Agent Setup

Docker Host will act as a Jenkins agent.

1. Create a Jenkins Agent Node

- Go to:
  Manage Jenkins → Manage Nodes and Clouds → New Node
- **Name:**docker-host
- **Type:**Permanent Agent

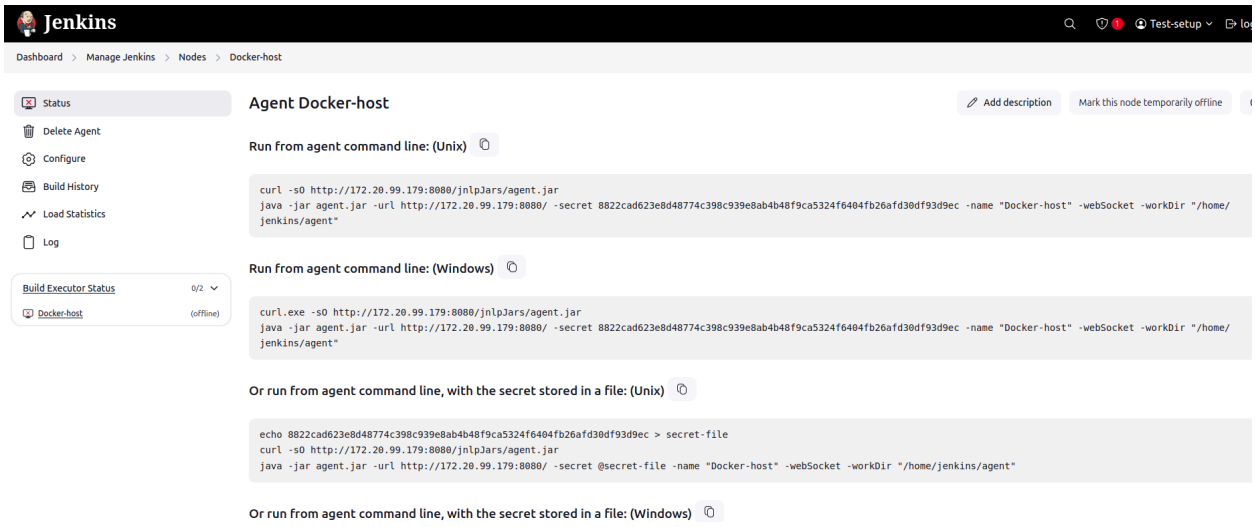
- Click **OK**

2. Configure the Agent Node

- **# of Executors:**2
- **Remote root directory:**/home/jenkins/agent
- **Labels:**docker-host
- **Launch method:**Launch agent by connecting it to the controller

Click **Save**

Once the agent is created, you'll be able to see its detailed configuration and connection instructions.



Docker Host Setup for Jenkins Agent

1. Login to Docker Host

On the Docker host, create the directory for the Jenkins agent:

sudo mkdir -p /home/jenkins/agent

```
cd /home/jenkins/agent
```

2. Download the Jenkins Agent .jar from Jenkins Master

Get the agent .jar file from your Jenkins master:

```
wget http://<jenkins-master>:8080/jnlpJars/agent.jar
```

3. Get the Secret for Agent Connection

Retrieve the secret by running the agent command on the Jenkins master:

```
echo 8822cad623e8d48******************************** > secret-file
```

4. Create Systemd Service for Jenkins Agent

Create a jenkins-agent.service systemd service file:

```
sudo vim /etc/systemd/system/jenkins-agent.service
```

Add the following configuration to the file:

```
[Unit]
Description=Jenkins JNLP Agent
After=network.target

[Service]
User=root
WorkingDirectory=/home/jenkins/agent
Environment="http_proxy=http://192.168.100.100:3128/"
Environment="https_proxy=http://192.168.100.100:3128/"
ExecStart=/usr/bin/java -jar /home/jenkins/agent/agent.jar \
  -url http://<masterip>:8080/ \
  -secret @/home/jenkins/agent/secret-file \
  -name "Docker-host" \
  -webSocket \
  -workDir "/home/jenkins/agent"
Restart=always

[Install]
WantedBy=multi-user.target
```

5. Enable and Start the Jenkins Agent Service

After creating the service file, reload systemd, enable, and start the Jenkins agent:

```
sudo systemctl daemon-reexec
sudo systemctl daemon-reload
sudo systemctl enable jenkins-agent
sudo systemctl start jenkins-agent
```

Check the status of the Jenkins agent:

```
systemctl status jenkins-agent
```

Test Setup Details

- **Package:** Imagemagick
- **SCM:**https://github.com/Configtm/BS_Imagemagick.git
- **Jenkins Master:**http://10.65.156.94:8080
- **JFrog:**http://10.65.150.52:8082
- **Docker Host:**10.65.149.232